Feature Engineering and Exploratory Data Analysis (EDA) Using Pandas, NumPy, and Matplotlib

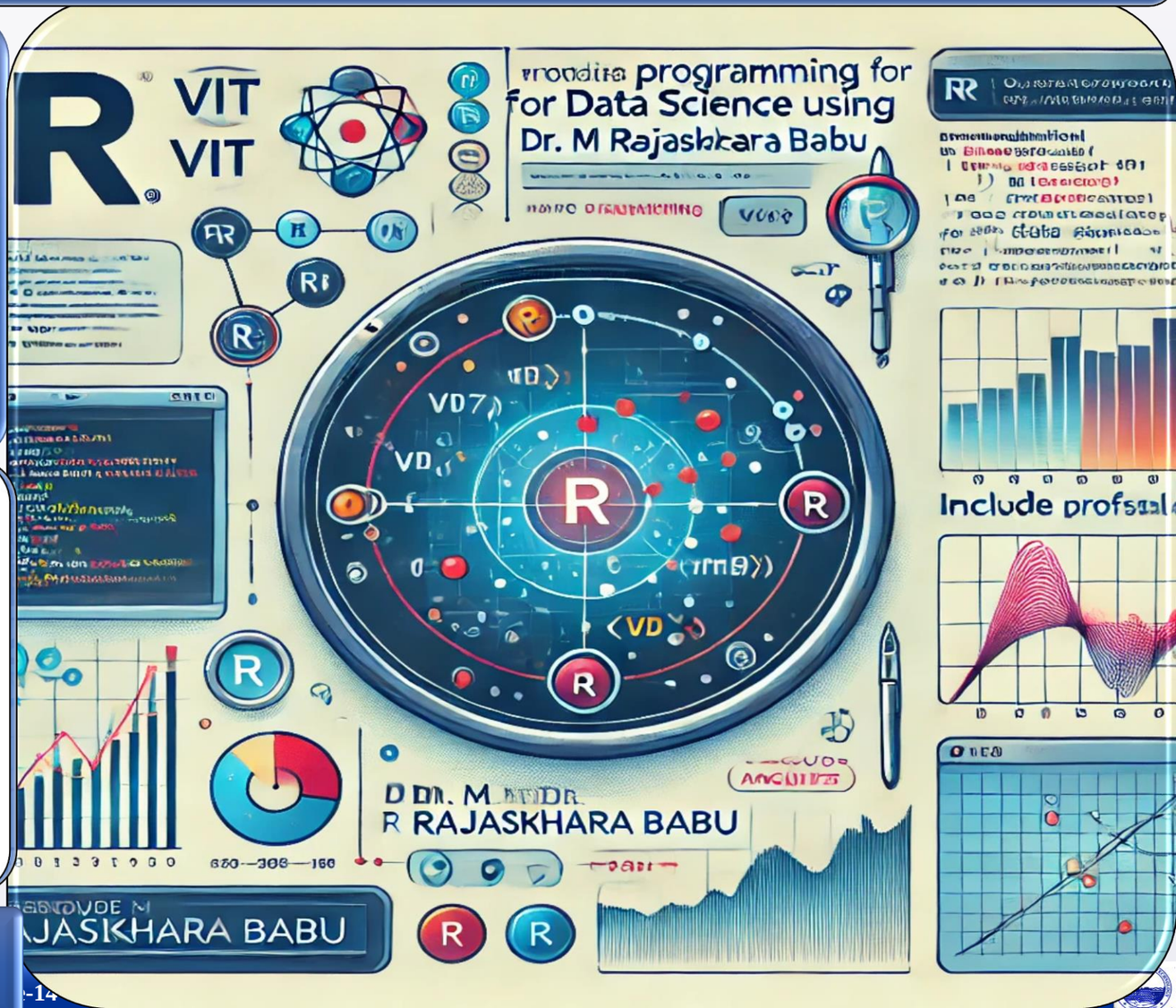
by

Dr M Rajasekhara Babu

School of

Computer Science and Engineering

https://www.rajababu.org



VELLORE INSTITUTE OF TECHNOLOGY
VIT
உழைப்பே உயர்வு தரும்

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India

Objectives & Teaching Learning Material

To explain feature engineering techniques such as logarithmic transformation and feature binning for improving data quality and model performance

To demonstrate Exploratory Data Analysis techniques including correlation analysis and data visualization for understanding datasets

To illustrate how engineered features and visual analytics can be used to discover patterns, relationships, and insights from data

Teaching + Learning Material: LCD, White board Marker, Presentation slides

Learning outcomes

Upon successful completion of this lecture, students will be able to:

Apply feature engineering techniques such as logarithmic transformation and feature binning to prepare data for analysis and machine learning.

Perform exploratory data analysis using correlation matrices and statistical techniques to understand variable relationships

Create and interpret visualizations such as scatter plots to identify patterns, trends, and predictive insights in datasets

Session Plan

Time in minutes	Topic	Learning Aid & Methodology	Faculty Approach	Student Activity	Skill and Competency Developed
05	Re-Cap	Quiz Presentation	Questions Organizes	Answers Identifies	Knowledge Understanding
10	Introduction to Feature Engineering and EDA	Presentation	Presentation	Listens	Remembering
10	Feature Transformation and Binning	Explains	Explains	Notes	Understanding
15	Correlation Analysis and Interpretation	Case Study	Organizes	Observes	Applying
10	Data Visualization and Pattern Discovery	Discussion	Facilitates	Answers	Analyzing

What is Feature Engineering?

Feature Engineering is the process of **creating, modifying, or transforming variables (features)** to improve data analysis and machine learning performance.

Original Features

- Age
- Salary
- Experience

Engineered Features

- Age Group
- Log(Salary)
- Experience Level

Model Accuracy

Improves prediction performance

Simplification

Compresses complex data

Hidden Patterns

Reveals latent relationships

What is Exploratory Data Analysis (EDA)?

EDA is the process of **examining datasets to understand their characteristics** before building predictive models.

Objectives

- Understand data distribution
- Detect patterns and outliers
- Discover relationships
- Generate hypotheses

Common Techniques

- Summary statistics
- Correlation analysis
- Histograms & scatter plots
- Box plots

Problem Statement Feature Engineering, Exploratory Data Analysis:

Create a DataFrame with columns **Feature1**, **Feature2**, and **Target containing random data**. Perform the following operations: **create a new feature that is the logarithm of Feature1,bin Feature2 into three categories (low, medium, high)**, and **calculate the correlation matrix of the DataFrame**. Finally, **create a scatter plot of Feature1 vs. Feature2 colored by Target**, and interpret any visible patterns.

Problem Statement Feature Engineering, Exploratory Data Analysis:

Create a DataFrame with columns Feature1, Feature2, and Target containing random data. Perform the following operations:

1. **Create a new feature that is the logarithm of Feature1.**
2. **Bin Feature2 into three categories (Low, Medium, High).**
3. **Calculate the correlation matrix of the DataFrame.**
4. **Create a scatter plot of Feature1 vs Feature2 colored by Target.**
5. **Interpret any visible patterns in the scatter plot.**

Steps / Procedure

Step 1: Import the required libraries: Pandas, NumPy, and Matplotlib.

Step 2: Generate random values for **Feature1**, **Feature2**, and **Target**.

Step 3: Create a DataFrame using the generated data.

Step 4: Create a new feature called **Log_Feature1** using the logarithm of Feature1.

Step 5: Bin **Feature2** into three categories: Low, Medium, and High.

Step 6: Calculate the correlation matrix of all numerical features.

Step 7: Create a scatter plot of Feature1 and Feature2, using Target values as colors.

Step 8: Display the transformed DataFrame, correlation matrix, and scatter plot.

Step 9: Interpret the relationship between variables.

Step 1: Libraries Required

Three core Python libraries power our Feature Engineering and EDA workflow:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Pandas

Data manipulation — loading, cleaning, and transforming tabular data.

NumPy

Numerical operations — arrays, random data generation, and math functions.

Matplotlib

Data visualization — scatter plots, histograms, and custom charts.

Step 2 &3: Generate random values &Creating the frame

We generate a synthetic dataset using NumPy's random functions with a fixed seed for reproducibility.

`np.random.seed()`

- is a function used to initialize the pseudo-random number generator (PRNG) in NumPy.
- Purpose
  - Initializes NumPy's random number generator.
  - Ensures that the same random numbers are generated every time the program runs.
  - Makes experiments reproducible

What Happens Without a Seed?

```
import numpy as np
print(np.random.randint(1,100,5))
#Run1: [32 18 45 91 73]
#Run2: [67 84 12 53 24]
```

What Happens With a Seed?

```
import numpy as np
np.random.seed(42)
print(np.random.randint(1,100,5))
#Run1: [52 93 15 72 61]
#Run2: [52 93 15 72 61]
```

Step 2 &3: Generate random values &Creating the frame

Features (Inputs)

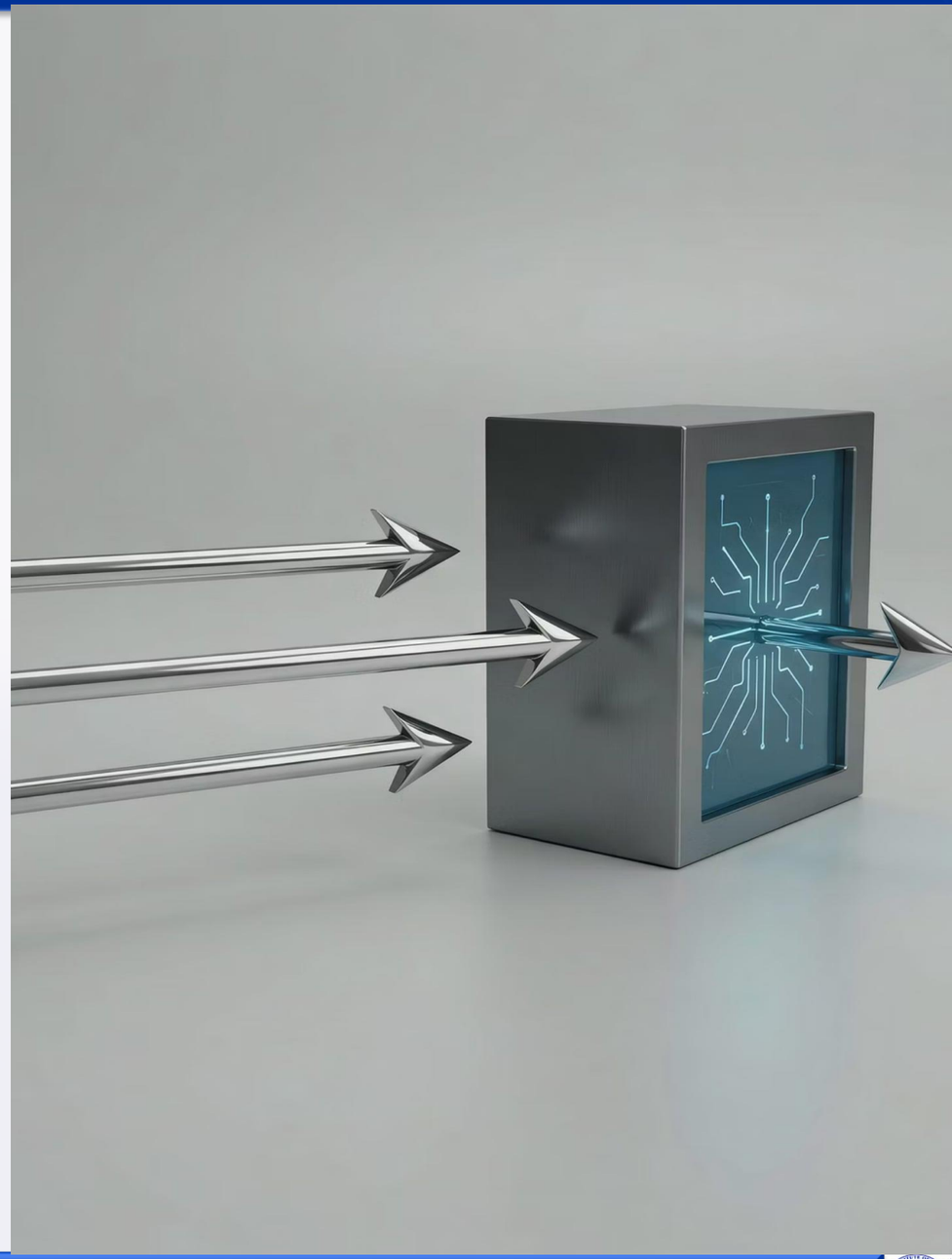
Variables used as inputs to the model.

- Feature1 → e.g., Age
- Feature2 → e.g., Income

Target (Output)

The variable the model tries to predict.

- Target → e.g., Purchase (0 or 1)



Machine learning models learn the relationships between features and the target variable.

Step 2 &3: Generate random values &Creating the frame

Code

```
#Step2: Generate random values for Feature1, Feature2, and
Target

np.random.seed(42)
data = {
    'Feature1': np.random.randint(1, 100, 20),
    'Feature2': np.random.randint(10, 200, 20),
    'Target':   np.random.randint(0, 2, 20)
}

#Step2: Create a DataFrame using the generated data.
df = pd.DataFrame(data)
```

Step 4: Log Transformation

A **logarithmic transformation** converts a feature value *X* to *log(X)*, compressing large values and reducing skewness.

```
Step 4: Create a new feature called Log_Feature1 using the logarithm of Feature1.
df['Log_Feature1'] = np.log(df['Feature1'])
```

Before Transformation

- 10 → 100 → 1,000 → 10,000
- Large gap between values

After Transformation

- 2.3 → 4.6 → 6.9 → 9.2
- Reduced variation

Reduce Skewness

Compress Large Values

Improve Distribution

Model Stability

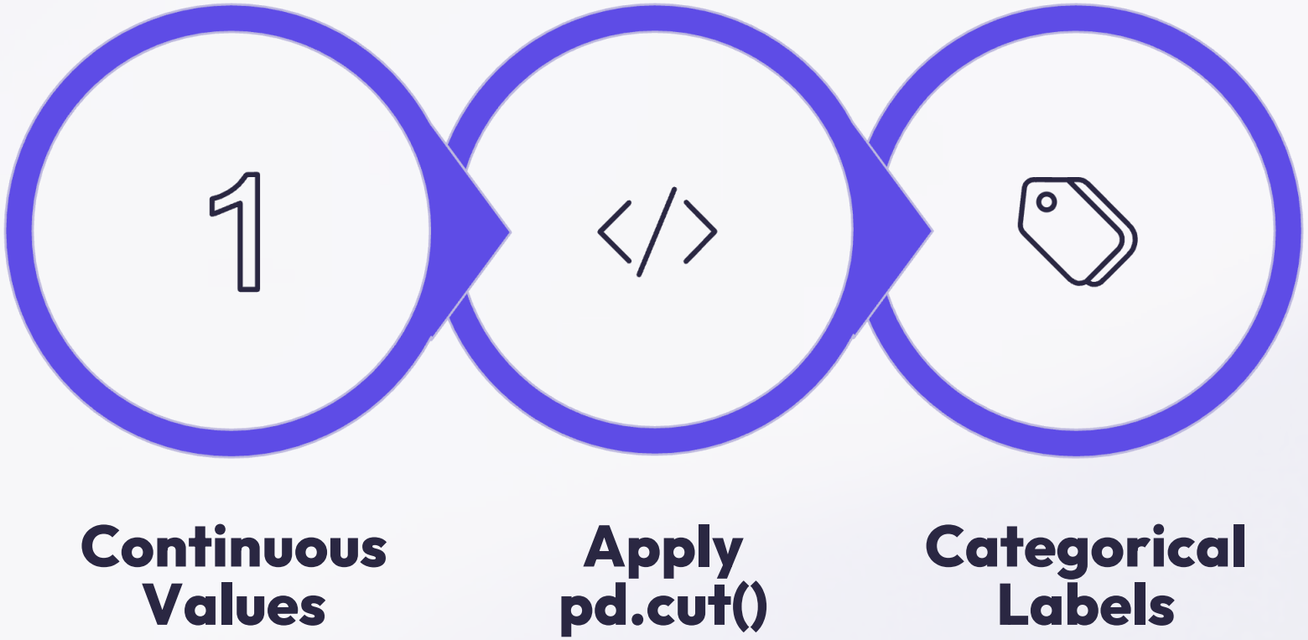
Step 4: Log Transformation — Example Output

Feature1	Log_Feature1
10	2.30
50	3.91
90	4.50

Large values become more manageable. Log transformation is widely applied to **sales data, population figures, financial transactions,** and **sensor readings** — any domain where values span several orders of magnitude.

Step5: Feature Binning

Binning converts continuous numerical values into discrete groups or categories, simplifying analysis and improving interpretability.



Binning reduces noise, supports classification tasks, and enables clearer business decision-making — for example, converting raw income into Low, Middle, and High Income bands.

Step 5: Types of Binning & Code

Equal Width Binning

Divides the range into equal-sized intervals: Low, Medium, High.

Equal Frequency Binning

Each group contains an equal number of observations.

Feature2	Category
25	Low
110	Medium
180	High

Pandas Code

```
df['Feature2_Category'] =
pd.cut
(
    df['Feature2'],
    bins=3,
    labels=['Low','Medium','High']
)
```

Step 6: Correlation Analysis

Correlation measures the strength and direction of the relationship between two variables. The correlation coefficient *r* ranges from −1 to +1.

Correlation Value	Meaning
+1	Perfect Positive
+0.7 to +0.99	Strong Positive
+0.3 to +0.69	Moderate Positive
0	No Relationship
−0.3 to −0.69	Moderate Negative
−0.7 to −0.99	Strong Negative
−1	Perfect Negative

Step 6: Computing the Correlation Matrix

A **correlation matrix** shows pairwise correlations among all selected variables at once — a key EDA tool.

```
corr_matrix =
df[
    ['Feature1', 'Feature2', 'Target', 'Log_Feature1']
  ].corr()
print(corr_matrix)
```

	Feature1	Feature2	Target	Log_Feature1
Feature1	1.00	0.25	0.12	0.97
Feature2	0.25	1.00	0.05	0.21
Target	0.12	0.05	1.00	0.10
Log_Feature1	0.97	0.21	0.10	1.00

Step 6: Interpreting the Correlation Matrix

Feature1 vs Log_Feature1 → 0.97

Very Strong Positive Relationship — expected, since Log_Feature1 is derived directly from Feature1.

Feature1 vs Feature2 → 0.25

Weak Positive Relationship — the two features share little linear association.

Feature2 vs Target → 0.05

Almost No Relationship — Feature2 has very low predictive power over the Target.

Step 7: Data Visualization

Understand Patterns

Identify Trends

Discover Relationships

Detect Outliers

Communicate Findings

Visualization transforms raw numbers into intuitive insights, making it easier to spot what statistics alone might miss.

Step 7: Scatter Plot — Code & Color Mapping

A **scatter plot** displays the relationship between two numerical variables. Each point represents one observation; color encodes the Target class.

```
plt.scatter(
    df['Feature1'],
    df['Feature2'],
    c=df['Target'],
    cmap='viridis',
    s=100
)
plt.xlabel("Feature1")
plt.ylabel("Feature2")
plt.show()
```

Color Mapping

- Target = 0 → Color A
- Target = 1 → Color B

Useful For

- Classification problems
- Cluster analysis
- Pattern recognition

Pattern Interpretation in Scatter Plots

Positive Correlation

Points move upward: Feature1 ↑ → Feature2 ↑. A positive relationship exists between the two variables.

Negative Correlation

Points move downward: Feature1 ↑ → Feature2 ↓. An inverse relationship exists.

Clusters

Well-separated clusters → strong predictive features. Overlapping clusters → weak predictive power.

- Since our dataset is randomly generated, expect no strong pattern, weak correlations, and significant overlap between Target classes — typical of synthetic data.

Complete Program

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
data = {
 'Feature1': np.random.randint(1, 100, 20),
 'Feature2': np.random.randint(10, 200, 20),
 'Target': np.random.randint(0, 2, 20)
}
df = pd.DataFrame(data)

df['Log_Feature1'] = np.log(df['Feature1'])
df['Feature2_Category'] = pd.cut(
 df['Feature2'], bins=3, labels=['Low','Medium','High'])

corr_matrix = df[
 ['Feature1','Feature2','Target','Log_Feature1']].corr()
print(corr_matrix)

plt.scatter(df['Feature1'], df['Feature2'],
 c=df['Target'], cmap='viridis')
plt.show()
```

Real-World Applications

Banking

Customer segmentation and credit risk analysis.

Healthcare

Disease prediction and patient classification.

Retail & Marketing

Sales prediction and customer behavior analysis.

Artificial Intelligence

Feature selection and data preparation for ML models.

Summary & Practice Exercise

Topics Covered

- Feature Engineering & Transformation
- Logarithmic Transformation
- Feature Binning
- Exploratory Data Analysis
- Correlation Matrix
- Scatter Plot Visualization
- Pattern & Relationship Discovery

Practice Exercise

Create a DataFrame with **Marks**, **Attendance**, and **Result**, then:

1. Create Log(Marks)
2. Bin Attendance into Low, Medium, High
3. Compute the Correlation Matrix
4. Create a Scatter Plot
5. Interpret the Results

Summary

- **Introduction to Feature Engineering**
  - Concept and importance of feature engineering
  - Benefits of creating transformed features
  - Improving model performance through feature transformation
- **Introduction to Exploratory Data Analysis (EDA)**
  - Purpose and objectives of EDA
  - Understanding distributions, patterns, and relationships
  - Common EDA techniques
- **Dataset Creation and Preparation**
  - Using Pandas, NumPy, and Matplotlib
  - Generating random datasets
  - Understanding features and target variables
  - Importance of reproducibility using random seeds
- **Feature Transformation Techniques**
  - Logarithmic transformation
  - Feature engineering using derived variables
  - Benefits of reducing skewness and improving distributions
- **Feature Binning**
  - Converting continuous values into categories
  - Equal-width and equal-frequency binning
  - Creating Low, Medium, and High categories using pd.cut()
- **Correlation Analysis**
  - Understanding correlation coefficients
  - Computing correlation matrices
  - Interpreting relationships among variables
- **Data Visualization**
  - Scatter plot creation using Matplotlib
  - Color mapping using target variables
  - Pattern recognition and cluster analysis
- **Pattern Interpretation and Applications**
  - Identifying positive and negative correlations
  - Understanding clusters and overlaps
  - Real-world applications in banking, healthcare, retail, and AI